

# Monitor B3

## Monitor B3 — Guia de implantação passo a passo

Sep 30, 2026 · @Vahid Nikoofard

### Visão geral

A implantação monta três peças, nesta ordem: o banco no Supabase, o pipeline Python agendado no GitHub Actions e o frontend em Streamlit. Cada etapa termina com uma verificação; só avance quando ela passar.

[embedded content: arquitetura do Monitor B3 · 5 peças]

Só o GitHub Actions, com a chave secreta, escreve nas tabelas de mercado; o frontend usa a chave publishable e fica sujeito às regras de RLS.

**O que é o GitHub Actions.** É um computador emprestado pelo GitHub. No horário marcado (ou quando você aperta um botão), ele liga uma máquina Linux nova, copia o seu repositório, executa os comandos escritos num arquivo de “receita” e desliga. Nada fica salvo nessa máquina: por isso o pipeline grava tudo no Supabase.

Vocabulário que aparece nas telas do GitHub:

| Termo          | O que é, neste projeto                                                                      |
|----------------|---------------------------------------------------------------------------------------------|
| Repositório    | A pasta do projeto hospedada no GitHub                                                      |
| Workflow       | A receita: o arquivo <code>.github/workflows/pipeline-diario.yml</code>                     |
| Run (execução) | Cada vez que a receita roda, agendada ou manual                                             |
| Job            | Um bloco de trabalho dentro da execução; aqui há um só, pipeline                            |
| Step (passo)   | Cada comando do job: instalar Python, rodar testes, rodar o pipeline                        |
| Runner         | A máquina virtual que executa o job                                                         |
| Secret         | Um valor guardado com segurança (as chaves do Supabase), nunca visível no código nem no log |

Tempo estimado: 15 minutos para o Supabase, 20 minutos para o GitHub, 5 minutos para a primeira execução e 15 minutos para publicar o frontend.

## Antes de começar

Você precisa de três contas gratuitas e da pasta do projeto já testada localmente.

☐

Conta no [GitHub](#)

☐

Conta no [Supabase](#) (entrar com a conta do GitHub simplifica)

☐

Conta no Streamlit Community Cloud, para o frontend (entra-se com a conta do GitHub)

☐

Pasta monitor-b3-aula extraída do zip mais recente

☐

`python -m tests.teste_offline` terminando com OK na sua máquina

☐

`python -m pipeline.main --teste-local` mostrando o ranking com dados reais (não grava nada)

Confira que a pasta oculta `.github` existe. No terminal, dentro de `monitor-b3-aula`, rode `ls -a` (no PowerShell do Windows, dir `-Force`): devem aparecer `.github`, `.gitignore`, `.streamlit`, `app`, `frontend`, `pipeline`, `sql`, `tests`, `README.md` e `requirements.txt`.

## Etapa 1 — Supabase

Ao final desta etapa, o banco tem 8 tabelas, 2 views, a função `montar_carteira` e o bucket `avatares`, e você tem em mãos a URL do projeto e a chave secreta. A construção passo a passo, pela interface, está no guia **Monitor B3 — Construindo o banco no Supabase**; aqui basta o script completo.

1. No painel do Supabase, clique em **New project**. Nome: `monitor-b3`; região: **South America (São Paulo)**; plano: **Free**. Crie uma senha do banco e guarde-a.
2. Aguarde a criação, que leva de 1 a 2 minutos.
3. Abra o **SQL Editor**, clique em **New query**, cole o conteúdo inteiro de `sql/01_schema.sql` e clique em **Run**. O resultado esperado é `Success`. No `rows returned`. Rode esse arquivo **uma única vez**.
4. Verifique, numa nova consulta:

```
select table_name, table_type
from information_schema.tables
where table_schema = 'public'
order by table_type, table_name;
```

Devem aparecer 8 linhas **BASE TABLE** (ativos, carteira\_itens, carteiras, modelos, precos\_diarios, previsoes, profiles, setores) e 2 linhas **VIEW** (v\_desempenho\_carteiras, v\_ranking\_atual). A tabela ativos já tem 24 ações. No menu **Storage**, deve existir o bucket avatares.

5. Copie a **URL do projeto**, no formato `https://xxxxxxx.supabase.co`. Ela aparece no botão **Connect** no topo do painel e nas configurações de API do projeto.
6. Copie a **chave secreta**: em **Project Settings → API Keys**, na seção de secret keys, revele e copie a chave que começa com `sb_secret_`. Se o seu projeto só mostrar as chaves antigas (aba Legacy), a `service_role` também funciona, mas o Supabase vai descontinuí-la até o fim de 2026.
7. Para a aula, desative a confirmação por e-mail em **Authentication**, nas configurações do provedor **Email** (opção *Confirm email*). Assim, as pessoas podem criar contas de teste sem precisar abrir a caixa de entrada.

A chave secreta ignora todas as regras de segurança (RLS) do banco. Ela só vai para os secrets do GitHub: nunca no código, no frontend ou num e-mail. O frontend usa outra chave, a `publishable`, que o Lovable configura sozinho.

## Etapa 2 — Repositório no GitHub

Ao final desta etapa, o repositório tem na raiz a pasta `.github` e a aba **Actions** mostra o workflow “Pipeline diário”. Há dois caminhos: pelo navegador (mais simples) ou pelo terminal (melhor para atualizações frequentes).

### Caminho A — pelo navegador

1. No GitHub, clique em **New repository**. Nome: `monitor-b3-aula`; visibilidade: **Public** (execuções gratuitas e ilimitadas). Não marque a opção de criar README: o projeto já tem um.
2. Na página do repositório vazio, clique no link **uploading an existing file**.
3. No gerenciador de arquivos, abra a pasta `monitor-b3-aula` e mostre os arquivos ocultos (**Ctrl+H** no gerenciador do Ubuntu; no Explorador de Arquivos do Windows, menu Exibir → Mostrar → Itens ocultos). Selecione o **conteúdo** da pasta, não a pasta em si: `.github`, `.gitignore`, `.streamlit`, `app`, `frontend`, `pipeline`, `sql`, `tests`, `README.md` e `requirements.txt`. Se existir uma pasta `__pycache__`, deixe-a de fora.
4. Se você criou o arquivo `.streamlit/secrets.toml` para testar o app, não o envie: pelo navegador, o `.gitignore` não vale.

5. Arraste a seleção para a página do GitHub, espere o upload e clique em **Commit changes**.

O erro mais comum aqui é arrastar a pasta monitor-b3-aula inteira. Aí tudo fica dentro de uma subpasta, a .github deixa de estar na raiz e o GitHub não encontra o workflow.

## Caminho B — pelo terminal

O GitHub não aceita mais a senha da conta no terminal. O jeito mais simples de autenticar é a ferramenta oficial gh, que também cria o repositório:

### No Linux (Ubuntu):

```
sudo apt install gh
gh auth login           # escolha GitHub.com, HTTPS e login pelo navegador
cd monitor-b3-aula
git init
git add .
git commit -m "Primeira versão do Monitor B3"
gh repo create monitor-b3-aula --public --source=. --push
```

Nesse caminho o .gitignore já exclui as pastas \_\_pycache\_\_. Se for a primeira vez que usa o git nesta máquina, antes do commit rode git config --global user.name "Seu Nome" e git config --global user.email "seu@email".

**No Windows:** abra o **PowerShell** (menu Iniciar → digite PowerShell). Instale o Git e o GitHub CLI com o winget, que já vem no Windows 10 e 11:

```
winget install --id Git.Git -e
winget install --id GitHub.cli -e
```

Depois da instalação, **feche e abra o PowerShell de novo**: só assim os comandos git e gh passam a ser reconhecidos. Se o winget não existir no computador, baixe os instaladores em [git-scm.com](https://git-scm.com) e [cli.github.com](https://cli.github.com). Em seguida, os mesmos passos do Linux:

```
gh auth login           # escolha GitHub.com, HTTPS e login pelo navegador
cd $HOME\Downloads\monitor-b3-aula # ajuste para a pasta onde você extraiu o zip
git init
git add .
git commit -m "Primeira versão do Monitor B3"
gh repo create monitor-b3-aula --public --source=. --push
```

Os comandos git config do parágrafo acima são iguais no Windows. Se o caminho da pasta tiver espaços, coloque-o entre aspas: cd "C:\Users\Ana\Meus arquivos\monitor-b3-aula".

## Verificação



Na página inicial do repositório, a pasta `.github` aparece na lista, ao lado de `pipeline` e `sql`



A aba **Actions** mostra “Pipeline diário” no menu lateral

## Etapa 3 — Entendendo o workflow

O arquivo `.github/workflows/pipeline-diario.yml` responde a três perguntas: **quando** rodar, **onde** rodar e **o que** rodar. Não é preciso alterá-lo; esta etapa é só leitura.

```
name: Pipeline diário

on:
  schedule:
    # 01:00 UTC de terça a sábado = 22:00 em Brasília de segunda a sexta,
    # depois do fechamento da B3.
    - cron: "0 1 * * 2-6"
  workflow_dispatch:          # cria o botão "Run workflow" na aba Actions
  inputs:
    primeira_vez:
      description: "Enviar todo o histórico de preços (só na primeira execução)"
      type: boolean
      default: false

jobs:
  pipeline:
    runs-on: ubuntu-24.04      # uma máquina Linux nova a cada execução (versão
                              # fixa)
    timeout-minutes: 30
    steps:
      - name: Copiar o repositório
        uses: actions/checkout@v6

      - name: Instalar o Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.12"
          cache: pip

      - name: Instalar as bibliotecas
        run: pip install -r requirements.txt

      - name: Teste sem internet
        run: python -m tests.teste_offline

      - name: Rodar o pipeline
        env:
          SUPABASE_URL: ${ secrets.SUPABASE_URL }
          SUPABASE_SERVICE_ROLE_KEY: ${ secrets.SUPABASE_SERVICE_ROLE_KEY }
```

```

run: |
  if [ "${{ inputs.primeira_vez }}" = "true" ]; then
    python -m pipeline.main --primeira-vez
  else
    python -m pipeline.main
  fi

```

| Trecho                  | O que faz                                                                                                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| name                    | Nome exibido na aba Actions                                                                                                                                      |
| schedule + cron         | Agenda: minuto 0, hora 1, de terça a sábado. O GitHub usa o horário UTC, então isso equivale a 22h de segunda a sexta em Brasília, depois do fechamento da bolsa |
| workflow_dispatch       | Cria o botão <b>Run workflow</b> , para rodar manualmente; inputs.primeira_vez vira uma caixa de seleção nesse botão                                             |
| runs-on: ubuntu-24.04   | A máquina: um Ubuntu 24.04 novo a cada execução, apagado no final. A versão fica fixa para uma atualização do GitHub não mudar o ambiente sem aviso              |
| timeout-minutes: 30     | Aborta se passar de 30 minutos (uma execução normal leva 2 a 4 minutos)                                                                                          |
| Copiar o repositório    | Traz o código do GitHub para a máquina                                                                                                                           |
| Instalar o Python       | Instala o Python 3.12 e guarda as bibliotecas em cache para as próximas execuções                                                                                |
| Instalar as bibliotecas | O mesmo pip install que você rodou na sua máquina                                                                                                                |
| Teste sem internet      | Se o teste falhar, a execução para aqui e nada é gravado no Supabase                                                                                             |
| Rodar o pipeline        | Entrega os secrets ao Python como variáveis de ambiente e roda as 5 etapas, com ou sem o histórico completo                                                      |

Dois detalhes do agendamento: o GitHub pode atrasar execuções agendadas em alguns minutos nos horários de pico, e elas só rodam a partir da branch principal (main).

## Etapa 4 — Secrets e primeira execução

Ao final desta etapa, a primeira execução termina com um check verde e o Supabase fica com 3 anos de preços das 24 ações, a primeira versão do modelo e o ranking do dia.

## Cadastrar os secrets

1. No repositório, abra **Settings → Secrets and variables → Actions** e clique em **New repository secret**.
2. Crie os dois secrets abaixo, com os nomes **exatamente** assim (maiúsculas e sublinhados):

| Nome                      | Valor                                                                              |
|---------------------------|------------------------------------------------------------------------------------|
| SUPABASE_URL              | A URL do projeto, <code>https://xxxxxxx.supabase.co</code>                         |
| SUPABASE_SERVICE_ROLE_KEY | A chave secreta <code>sb_secret_...</code> (ou a <code>service_role</code> antiga) |

O nome do segundo secret é o que o código espera, mesmo quando o valor é a chave nova. Depois de salvo, um secret não pode mais ser visualizado, só substituído. Nos logs, qualquer secret aparece como `***`.

## Rodar pela primeira vez

1. Abra a aba **Actions**. Se aparecer um aviso pedindo para habilitar os workflows, confirme.
2. No menu lateral, clique em **Pipeline diário** e depois em **Run workflow**, à direita.
3. Mantenha a branch principal (`main` ou `master`, conforme o repositório), **marque a caixa “Enviar todo o histórico de preços”** e clique no botão verde **Run workflow**.
4. Em alguns segundos surge uma execução com um círculo amarelo. Clique nela, depois no job **pipeline**, para ver os passos. Cada passo pode ser expandido; o passo “Rodar o pipeline” mostra o mesmo log que você viu na sua máquina, etapa por etapa.

Duração esperada: cerca de 1 minuto para instalar as bibliotecas, alguns segundos de teste e 1 a 3 minutos para o pipeline. Na primeira execução, ele envia cerca de 18 mil linhas de preços (24 ações × 3 anos); nas seguintes, só os últimos 10 pregões.

Check verde significa sucesso. Um X vermelho indica falha: abra o passo com o X, leia as últimas linhas do log e consulte a seção de solução de problemas.

A caixa “Enviar todo o histórico” só é necessária nesta primeira vez. Nas execuções agendadas, ela fica desmarcada.

## Etapa 5 — Conferir os dados no Supabase

Três consultas no SQL Editor confirmam que o pipeline gravou tudo. Rode uma de cada vez.

```
-- 1. Preços: 24 ações, cerca de 18 mil linhas, última data = último pregão
select count(distinct ticker) as acoes, count(*) as linhas, min(data), max(data)
from precos_diarios;

-- 2. Modelo publicado, com a métrica de qualidade
select versao, treinado_em, n_amostras, ic_validacao
from modelos
order by treinado_em desc;

-- 3. Ranking do dia
select posicao, ticker, setor, round(retorno_previsto::numeric, 4) as previsto
from v_ranking_atual
order by posicao
limit 10;
```

O bucket avatares continua vazio: ele só recebe arquivos quando alguém troca a foto pelo app.

A coluna `retorno_realizado` da tabela `previsoes` fica vazia nos primeiros dias. Ela é preenchida automaticamente quando se completam os 5 pregões de cada previsão.

## Etapa 6 — Frontend em Streamlit

Ao final desta etapa, o app está publicado num endereço `nome.streamlit.app`, com login, ranking do dia, página da ação, montagem de carteiras, página do modelo e perfil com foto, cada usuário vendo só as próprias carteiras. O código fica na pasta `app/` do mesmo repositório.

### Testar na sua máquina

Copie o modelo de chaves e preencha com a URL do projeto e a chave **publishable** (`sb_publishable_...`, em **Project Settings** → **API Keys** do Supabase). Esse arquivo está no `.gitignore` e não vai para o GitHub.

```
pip install -r app/requirements.txt
cp .streamlit/secrets.toml.example .streamlit/secrets.toml
streamlit run app/streamlit_app.py
```

No PowerShell do Windows, o comando `cp` funciona igual (é um apelido de `Copy-Item`); só troque as barras do caminho se preferir:

```
copy .streamlit\secrets.toml.example .streamlit\secrets.toml.
```

**Atenção: preencha a cópia, nunca o modelo.** O comando acima cria um arquivo novo, `.streamlit/secrets.toml`, e é **nele** que vão a URL e a chave reais. O `secrets.toml.example` vai para o GitHub de propósito e deve continuar só com os valores de exemplo (`https://xxxxxxx.supabase.co` e `sb_publishable_...`), para mostrar a quem clonar o projeto quais chaves preencher. O app lê apenas o `secrets.toml`.

Para conferir que a cópia está sendo ignorada pelo git, rode `git check-ignore -v .streamlit/secrets.toml`: a resposta deve mostrar a linha correspondente do `.gitignore`.

Se os valores reais acabarem no `.example` e forem para o GitHub, volte o arquivo aos valores de exemplo e faça um novo commit. Com a chave **publishable** o risco é baixo: ela foi feita para ser pública (todo navegador que abre o app a recebe), e a RLS continua protegendo os dados. Como ela fica no histórico do repositório, é um bom hábito trocá-la: em **Project Settings → API Keys**, crie uma nova publishable, atualize o `secrets.toml` e os Secrets do Streamlit Cloud, confira o app e só então apague a antiga. Com a chave **secreta**, trocar é obrigatório e imediato: ela ignora a RLS.

## Publicar no Streamlit Community Cloud

1. Acesse [share.streamlit.io](https://share.streamlit.io), entre com a conta do GitHub e autorize o acesso aos seus repositórios.
2. Clique em **Create app** e escolha publicar a partir de um repositório do GitHub. Repositório: `monitor-b3-aula`; branch: a principal do repositório (`main` ou `master`); arquivo principal: `app/streamlit_app.py`. Escolha o endereço do app.
3. Em **Advanced settings**, selecione Python 3.12 e, no campo **Secrets**, cole as duas linhas do seu `.streamlit/secrets.toml`, com a URL e a chave publishable.
4. Clique em **Deploy**. A primeira publicação leva 2 a 3 minutos; o Streamlit instala as bibliotecas de `app/requirements.txt`.

O app usa a chave **publishable**, nunca a secreta: depois do login, cada consulta vai ao banco com a identidade do usuário, e as políticas de RLS decidem o que ele vê. A cada `git push`, o app é atualizado sozinho.

No plano gratuito, um app sem acesso por 12 horas hiberna, e o primeiro visitante vê uma tela para acordá-lo, o que leva cerca de um minuto. Antes de uma aula, abra o app alguns minutos antes.

**Alternativa no-code.** O arquivo `frontend/prompt_lovable.md` gera um frontend equivalente no Lovable. Se usar esse caminho, conecte o seu Supabase ao projeto do Lovable logo no início (**More → Cloud → Already have a Supabase project?**), antes que ele ative o Lovable Cloud: depois disso, o projeto não aceita mais o seu Supabase.

## Teste de segurança com dois usuários

Esse teste mostra na prática o que as políticas de RLS fazem.

☐

Crie o usuário A, monte uma carteira e confira que ela aparece em **Minhas carteiras**

☐

Ainda como A, troque a foto em **Meu perfil**; no Supabase, em **Storage → avatares**, aparece uma pasta com o id de A

☐

Saia e crie o usuário B, com outro e-mail

☐

Confira que o usuário B vê o mesmo ranking do dia, mas a lista de carteiras dele está vazia

☐

No Supabase, em **Authentication → Users**, os dois usuários aparecem; na tabela profiles, cada um tem sua linha criada pelo trigger

## Rotina depois da implantação

A partir daqui o pipeline roda sozinho de segunda a sexta, às 22h. O acompanhamento se resume a olhar a aba **Actions** de vez em quando.

- **Falhas:** o GitHub envia um e-mail quando uma execução agendada falha. Na maioria das vezes é instabilidade do Yahoo Finance; nesse caso, abra a execução e clique em **Re-run jobs**.
- **Rodar fora do horário: Run workflow**, com a caixa “Enviar todo o histórico” desmarcada. Rodar duas vezes no mesmo dia não duplica nada.
- **Incluir ou retirar ações:** é no banco, não no código. Insira uma linha na tabela ativos pelo Table Editor (com o setor em `setor_id`), ou mude ativo para false para o pipeline deixar de acompanhar uma ação. A mudança vale a partir da próxima execução.
- **Inatividade de 60 dias:** num repositório público sem nenhum commit por 60 dias, o GitHub desativa o agendamento. A aba Actions mostra um aviso com um botão para reativar; qualquer commit também reinicia a contagem. Vale lembrar disso nas férias.
- **Atualizar o código:** edite o arquivo direto no GitHub (ícone de lápis) ou envie pelo terminal com `git add`, `git commit` e `git push`. A próxima execução já usa a versão nova. Os parâmetros do modelo ficam em `pipeline/config.py`.
- **Supabase gratuito:** o projeto é pausado após um período sem uso. As gravações diárias devem mantê-lo ativo, mas confira o status no painel nas primeiras semanas.
- **Depois do teste:** religue a confirmação por e-mail (**Authentication**, provedor **Email**, opção *Confirm email*). Com ela desligada, um app público facilita a criação de contas em massa por scripts, que poderiam consumir a cota gratuita de armazenamento com fotos.
- **Chaves antigas:** se você usou a `service_role`, troque pela chave `sb_secret_` antes do fim de 2026. Basta atualizar o valor do secret; o código não muda.

## Solução de problemas

A maioria dos erros vem de um nome, uma chave ou uma pasta no lugar errado. Procure a mensagem do log na coluna da esquerda.

| Sintoma                                                      | Causa provável                                                                           | O que fazer                                                                          |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| O workflow não aparece na aba Actions                        | A pasta .github não está na raiz do repositório                                          | Conferir se não foi enviada a pasta monitor-b3-aula inteira; reenviar o conteúdo     |
| Defina as variáveis SUPABASE_URL e SUPABASE_SERVICE_ROLE_KEY | Secret não criado ou com outro nome                                                      | Conferir os dois nomes, letra por letra                                              |
| Invalid API key ou erro 401                                  | Foi copiada a chave publishable, ou a chave tem espaço sobrando                          | Copiar de novo a chave sb_secret_ e substituir o secret                              |
| A tabela 'ativos' está vazia                                 | O script SQL não rodou inteiro, ou rodou em outro projeto                                | Rodar 01_schema.sql no projeto cuja URL está no secret                               |
| relation "public.ativos" does not exist                      | O schema não foi criado no projeto da URL cadastrada                                     | Rodar 01_schema.sql no projeto certo                                                 |
| already exists ao rodar o schema                             | O schema já tinha sido criado                                                            | Nada a fazer; para recomeçar do zero, crie um projeto novo no Supabase               |
| Não foi possível baixar os preços do Yahoo Finance           | Instabilidade do Yahoo                                                                   | <b>Re-run jobs;</b> o código já tenta 3 vezes antes de desistir                      |
| XXXX3: sem dados no Yahoo – ignorado no log                  | O ticker mudou de código ou foi digitado errado em ativos                                | Corrigir ou desativar (ativo = false) a linha na tabela ativos                       |
| git push recusado mencionando workflow                       | O token não tem permissão para arquivos de workflow                                      | Autenticar com gh auth login                                                         |
| Execução agendada não rodou                                  | Atraso do GitHub ou agendamento desativado por inatividade                               | Aguardar; conferir se há aviso na aba Actions                                        |
| Frontend mostra listas vazias                                | Pipeline ainda não rodou, ou usuário não está logado                                     | Rodar o workflow; entrar com um usuário (a RLS só libera leitura a usuários logados) |
| Erro ao enviar a foto                                        | Arquivo maior que 2 MB, tipo não permitido ou caminho que não começa com o id do usuário | Conferir o arquivo; o caminho deve ser <id do usuário>/avatar                        |